



Yes. Grafana is a general-purpose visualization and dashboarding layer, not limited to “systems observability.” It works well for *any* time-series (and even tabular/log) data—including trading/market data—so long as your data is in (or can be queried from) a supported data source.

How this typically works for trading data

1. **Ingest/Store market/trading time series** in a database suited for time series or analytics:
 - Time series: **TimescaleDB (Postgres), InfluxDB, Prometheus/Mimir** (more metrics-oriented)
 - Analytics/OLAP: **ClickHouse, BigQuery, Snowflake**
 - Search/log style: **Elasticsearch/OpenSearch**
2. **Connect Grafana to that source** via a built-in data source plugin (or a custom plugin).
3. **Build dashboards** for prices, volumes, positions, P&L, risk metrics, latency, etc.
4. Optionally use **Grafana Alerting** for threshold/condition-based alerts (e.g., drawdown, spread widening, volatility spike).

Visualizations you can do (examples)

- Line/area charts for price, volume, VWAP, realized volatility
- Heatmaps for order book depth (depending on how you model data)
- Tables for trades/executions, top movers
- Correlation matrices (via plugins or transformations)
- Alert panels for risk limits, exposure, unusual activity

Important considerations for trading use cases

- **Candlesticks/OHLC:** Grafana can do financial-style charts via:
 - Built-in time series + transformations (sometimes enough), and/or
 - Community/enterprise panels/plugins designed for candlesticks/OHLC (depends on your Grafana version and plugin availability).
- **Latency & “real-time”:** Grafana can refresh frequently, but it’s not a trading terminal. If you need sub-second streaming like Bloomberg/Refinitiv-style terminals, you’ll need careful architecture (streaming pipelines, cache, downsampling) and realistic expectations.
- **Data volume/downsampling:** Tick data is huge—use:
 - Aggregations (1s/1m/5m bars), continuous aggregates (Timescale), materialized views, or OLAP stores (ClickHouse)
 - Retention policies and precomputed rollups
- **Security & entitlements:** Trading data often needs strict access control. Grafana supports orgs/teams/roles and can integrate with SSO; row-level security is typically enforced in the database layer.

Bottom line

Grafana is absolutely suitable for trading/financial time-series dashboards—many teams use it for market data monitoring, strategy KPIs, risk and P&L tracking, and operational views—provided you pair it with the right storage/query layer and handle aggregation/latency requirements appropriately.

If you tell me your expected frequency (tick vs 1s/1m bars), rough volume, and preferred storage (Postgres/Timescale, ClickHouse, Influx, etc.), I can suggest a concrete architecture and dashboard layout.

Can it be integrated as a separate service with a React frontend and a golang/python backend

T



Yes—Grafana can be integrated as a separate service alongside a React frontend and a Go/Python backend. The common patterns are:

1) Embed Grafana dashboards inside your React app (most common)

How

- Run Grafana as its own service (Docker/K8s/VM).
- Your React app displays specific dashboards/panels via **Grafana embedding** (typically an `<iframe>`), or by linking out.

Auth options

- **SSO** (OIDC/SAML/LDAP) so users log in once and Grafana respects the same identity provider.
- **Reverse proxy** in front of Grafana (e.g., Nginx) to handle auth and pass identity headers (Grafana “auth proxy” mode).
- **Anonymous + signed/controlled access** only if data is non-sensitive (often not acceptable for trading).

Pros/cons

- Fastest to implement; full Grafana UI features.
- UI control is limited (it “looks like Grafana” inside your app unless heavily themed).

2) Use Grafana only as a visualization engine; your app controls the experience

You keep your React UI fully custom and:

- Query your **Go/Python API** for data, and chart it with React charting libraries; or

- Use Grafana panels in a more “headless” way is limited—Grafana isn’t designed as a pure chart-rendering SDK.

Grafana does have HTTP APIs, but they’re mainly for **managing** dashboards, data sources, users, etc., not for being a backend chart renderer for a custom UI.

3) Provision dashboards and data sources from your backend (DevOps-friendly)

Your Go/Python backend (or CI/CD) can:

- Create/update dashboards via **Grafana HTTP API**
- Provision data sources/dashboards via **files** (Grafana provisioning)
- Manage folders/permissions

This is a good fit when dashboards are “productized” and version-controlled.

4) Expose data through your backend, and let Grafana read from it

If you don’t want Grafana to connect directly to your DB:

- Your Go/Python service can expose metrics/time series in a Grafana-friendly way:
 - **Prometheus metrics endpoint** (common if it’s aggregations/metrics)
 - A database/warehouse Grafana connects to (preferred for analytics)
 - Or build a **Grafana data source plugin** (more work, but gives tight control)

Suggested architecture (typical)

- **React:** your app + routes that embed dashboards for “Analytics / Monitoring”
- **Go/Python API:** business logic, entitlements, aggregation, trading domain services
- **Data store:** TimescaleDB/ClickHouse/Influx/etc.
- **Grafana:** dashboards + alerting
- **Auth:** OIDC (Keycloak/Auth0/Azure AD/etc.) shared between React and Grafana

Key design decision: where to enforce permissions?

For trading data, you usually want:

- **DB-level security** (row-level security / per-tenant schema), or
- A controlled analytics store per tenant, because embedding Grafana with broad DB credentials can leak data if not designed carefully.

If you share:

1. your auth setup (OIDC?),
2. whether you want iframe embedding or “native” charts,
3. your data store (Timescale/ClickHouse/etc.), I can recommend the best integration pattern and a secure auth/permissions approach.

Provide an architectural solution for a small business and local-first data storage, including parquet files and duckdb for analytical tasks



Below is a pragmatic **local-first “mini-lakehouse” architecture** for a small business that wants to store data locally (offline-capable), keep it in **Parquet files**, and use **DuckDB** for analytics—optionally surfacing dashboards in **Grafana** and/or your **React** UI.

1) High-level architecture (local-first)

Diagram <> Code

Copy

```

flowchart LR
    subgraph Devices[Local environment (single PC / small server / NAS)]
        A[React Frontend<br/>(Browser)] -->|HTTPS| B[Backend API<br/>Go or Python]
        B -->|read/write| S[(Operational Store<br/>SQLite)]
        B -->|ingest/transform/query| D[DuckDB Engine<br/>(embedded in backend)]
        D -->|reads/writes| L[(Parquet Data Lake<br/>Local folder)]
        B -->|optional embed| G[Grafana Service<br/>(local)]
        G -->|queries| D
    end

    subgraph OptionalSync[Optional sync/backup]
        X[Restic / Borg Backup<br/>External disk or S3-compatible]
        Y[Syncthing / rsync / NAS replication<br/>(file sync)]
    end

    S --> X
  
```



```
L --> X
```

```
L --> Y
```

Key idea:

- **SQLite** handles “operational” app needs (users, settings, small transactional tables, audit log pointers).
- **Parquet** is your durable analytical storage (append-friendly, portable, easy to back up).
- **DuckDB** is your analytical query engine (runs locally, reads Parquet directly, fast).

2) Storage model (recommended layout)

Use a simple folder-based lake that's easy to understand and back up:

- `/data/raw/` = immutable ingested data (as-received)
- `/data/curated/` = cleaned/normalized datasets
- `/data/marts/` = analytics-ready tables (KPIs, finance, inventory, trading-like time series, etc.)
- `/data/tmp/` = scratch space for transforms
- `/meta/` = small metadata DB (SQLite), config, last-ingest checkpoints

Example partitioning strategy (works well for time series + business data):

- Partition by `dataset + year/month/day` (and optionally `location_id`, `account_id`, etc.)

- Use Parquet files sized roughly **128–512MB** when possible (not mandatory for small setups, but helps long-term)
-

3) Data flow (ingest → curate → serve)

A) Ingestion (local-first)

Sources might be:

- CSV exports (accounting/POS)
- API pulls (banking, ecommerce)
- Local app events (sales, invoices, inventory updates)
- Streaming-like data (prices/trades) as periodic batches

Ingest pattern:

1. Land files in `/data/raw/<source>/<date>/...parquet`
2. Record a checkpoint in SQLite (e.g., last imported file hash / timestamp)
3. Never modify raw files after landing (append-only mindset)

B) Transformation / modeling (DuckDB)

Run periodic transforms (every hour/day) to create curated and mart tables:

- Clean types, deduplicate, standardize keys

- Join reference data (products, accounts, customers)
- Build KPI tables (daily revenue, margins, inventory turns, etc.)
- For time series: generate OHLCV bars, rolling metrics, alerts

You can implement transforms:

- Direct DuckDB SQL executed by backend
- Or `dbt-duckdb` if you want versioned SQL models (nice even for small teams)

C) Serving analytics

Two common patterns:

Pattern 1 (simplest): Backend as the query gateway

- React calls backend endpoints like `/reports/sales?from=...&to=...`
- Backend runs DuckDB queries and returns JSON
- You control security and “approved queries”

Pattern 2 (Grafana dashboards): Grafana connects to DuckDB

- Run Grafana locally and connect via a **DuckDB data source plugin** (if available/acceptable in your environment)
- Or: materialize marts into a small Postgres instance for Grafana if you prefer “standard” datasources
(this adds a service, but can simplify Grafana connectivity)

4) React + Go/Python backend (local-first friendly)

Backend responsibilities

- Authentication/roles (even if simple)
- Ingestion orchestration (file watcher or "Import" button)
- DuckDB query execution (parameterized, controlled)
- Optional: schedule jobs (hourly/daily transforms)
- Serve dashboards (Grafana embed) or native React analytics

Recommended backend approach

- **Go**: great for a single static binary, easy deployment for small businesses
- **Python (FastAPI)**: fastest iteration for analytics-heavy logic

Deployment options:

- Single machine: run backend + Grafana + storage folders locally
- Small office server/NAS: run as services; users connect over LAN
- Laptop-first: everything local, backups to external disk

5) Local-first syncing & backup (what actually works)

Backups (strongly recommended)

Use **restic** (or borg) to back up:

- `/data/**` (Parquet)
- `/meta/**` (SQLite + configs)

Targets:

- External USB drive (best for small business reliability)
- S3-compatible storage (optional)

Sync (optional, be careful)

If you need multi-device sync:

- Parquet files are great for file-based sync **if you keep them immutable**
- Tools: Syncthing / rsync / NAS replication

Rule of thumb: avoid two machines writing to the same Parquet "table partition" concurrently.

If you need multi-writer, use:

- per-device landing zones (e.g., `/data/raw/siteA/...` , `/data/raw/siteB/...`) then merge downstream.

6) Example "minimum viable stack" (small business)

Single office server (or NAS + small compute):

- Backend: Go or FastAPI (systemd service or Docker Compose)
 - Storage: local disk (mirrored if possible)
 - SQLite: operational metadata
 - Parquet folders: analytical store
 - DuckDB: embedded in backend for transforms + queries
 - Optional Grafana: dashboards for owners/managers
 - Backup: restic nightly to USB + weekly offsite
-

7) When to add a “real” database server

Stay with DuckDB + Parquet as long as:

- Data volume is moderate (GBs to low TBs depending on machine)
- Concurrency needs are low (a few users)
- You can accept “batch-style” updates

Consider adding Postgres/Timescale/ClickHouse only if:

- Many concurrent users need interactive dashboards all day
- You need strict multi-writer concurrency
- You need near-real-time streaming ingestion at scale

8) Next inputs I need to tailor this

1. Single machine vs small LAN server vs multiple sites?
2. Approx data size and growth (e.g., 5GB now → +1GB/month)?
3. “Time series like trading”: tick-level, per-second, per-minute bars?
4. Do you want Grafana embedded, or all dashboards built natively in React?

With those, I can propose a concrete folder schema, job schedule, and “mart tables” design (including Parquet partition keys) that will stay maintainable for a small business.

1. Single machine, 2. daily data < 100 MB 3. Time series per-minute 4. Grafana embedded as a separate analytical endpoint

T



Target architecture (single machine, <100MB/day, per-minute series, Grafana embedded)

Diagram <> Code

Copy

```
flowchart TB
    U[User Browser] --> R[React Frontend]

    R -->|HTTPS| API[Backend API (Go/Python)]
    API --> OPS["(SQLite - operational<br/>users, settings, ingest checkpoints)"]
    API --> DDB[DuckDB (embedded)]
    DDB --> PQ["(Local Parquet Lake<br/>raw/curated/marts)"]

    subgraph Analytics[Embedded analytics endpoint]
        API -->|reverse proxy /grafana| G[Grafana (local service)]
        G -->|DataSource| DS[Analytics datasource]
        DS --> API
    end

    subgraph Backup[Local-first durability]
        PQ --> B["Restic/Borg backup<br/>to external disk"]
        OPS --> B
    end
```

Why this fits your constraints

- **Single machine:** everything runs locally; no network dependency.
 - **<100MB/day + per-minute:** Parquet + DuckDB is more than sufficient; queries stay fast with basic partitioning.
 - **Grafana embedded:** Grafana is a separate service, but surfaced through your backend as a dedicated “analytics endpoint” (e.g., <https://app.local/grafana>).
-

1) Storage layout (simple “mini-lakehouse”)

Recommended folders:

- `data/raw/` – immutable landings (what you ingested)
- `data/curated/` – cleaned, typed, deduped
- `data/marts/` – analytics-ready aggregates for dashboards
- `data/tmp/` – scratch/compaction
- `meta/app.db` – SQLite (operational + ingest checkpoints)
- `meta/duckdb.duckdb` – optional persistent DuckDB file (or purely embedded)

Partitioning (practical and enough for your volume):

- Partition primarily by **date** (e.g., `dt=YYYY-MM-DD`)
- Keep `symbol` (or entity id) as a column; only sub-partition by symbol if you later have many symbols and slow scans.

Example paths:

- `data/raw/minute_bars/dt=2026-09-02/source=vendorA/part-0001.parquet`
 - `data/curated/minute_bars/dt=2026-09-02/part-0001.parquet`
 - `data/marts/kpi_pnl_daily/dt=2026-09-02/part-0001.parquet`
-

2) Data model (time series per-minute)

Curated “minute bars” dataset (typical fields):

- `ts` (timestamp, minute precision)
- `symbol` (text)
- `open`, `high`, `low`, `close` (double/decimal)
- `volume` (double/int)
- optional: `venue` , `currency` , `timezone` , `ingest_id`

DuckDB will query this directly from Parquet for ad-hoc analysis, but for Grafana you’ll usually expose:

- the raw/minute series (for charts)
 - a few “marts” tables (for fast dashboards): daily OHLC, rolling volatility, daily volume, etc.
-

3) DuckDB usage pattern (operationally reliable)

Ingestion

- Backend imports CSV/JSON/API payloads → writes **raw Parquet** (append-only).
- Record checkpoint in SQLite: last imported timestamp/file hash.

Curate job (scheduled)

- DuckDB reads raw Parquet → writes curated Parquet (typed, deduped).
- Keep curated datasets stable and query-friendly.

Mart job (scheduled)

- DuckDB computes aggregates used by dashboards:
 - daily summaries
 - rolling 30/90 day metrics
 - “top movers” lists
 - exception tables (missing data, spikes)

Scheduling (simple, single machine):

- Every 5–15 minutes: ingest new minute data (if applicable)
- Nightly: curate + marts rebuild for the day
- Weekly: compaction (merge small Parquet files into fewer larger ones)

4) Grafana integration as a separate embedded endpoint

Reverse proxy / embedding

- Run Grafana locally (e.g., `localhost:3000`)
- Your backend exposes it at `/grafana` :
 - `https://app.local/grafana/*` → proxy to `http://localhost:3000/*`
- React “Analytics” page embeds Grafana dashboards via `iframe` pointing to `/grafana/d/...`

Auth (recommended):

- Use Grafana **Auth Proxy** mode so your backend is the gatekeeper.
- Backend validates the user session, then injects a header (e.g., `X-WEBAUTH-USER`) to Grafana.

This keeps Grafana “separate service” but still controlled through your app.

5) How Grafana queries DuckDB/Parquet (2 viable options)

Option A (cleanest for your setup): Grafana → Backend Analytics API → DuckDB

Grafana uses a datasource that calls HTTP endpoints you provide (a dedicated analytics endpoint).

- Backend runs parameterized DuckDB queries and returns time series JSON.
- You fully control security, query shapes, and performance.

Typical endpoints:

- GET /analytics/series/minute-bars?symbol=ABC&from=...&to=...
- GET /analytics/kpis/daily?from=...&to=...

This is usually the best “local-first + embedded Grafana” approach because Grafana doesn’t need direct file/DB access.

Option B: Grafana queries a SQL datasource you publish

If you prefer SQL directly in Grafana:

- Materialize `marts` into a small **local Postgres** (or another supported DB) from DuckDB nightly.
- Grafana connects to Postgres and queries with SQL.

This adds one more service, but keeps Grafana very standard.

Given your volume (<100MB/day), **Option A** is typically simpler and more secure.

6) Backup & local-first durability

Minimum recommended:

- Nightly backup with **restic** (or borg) of:
 - `data/**` (Parquet)
 - `meta/**` (SQLite + config + optional duckdb file)

- Target: external USB drive (best), optionally plus an offsite copy.

Operational rule:

- Treat Parquet in `raw/` and `curated/` as **immutable** once written (makes backups/sync safe and avoids corruption issues).
-

7) Concrete “bill of materials” (single machine)

- React frontend (served by backend or separate static server)
 - Backend API (Go/Python)
 - ingestion + scheduling
 - DuckDB embedded for transforms/queries
 - reverse proxy to Grafana at `/grafana`
 - analytics endpoints for Grafana datasource (Option A)
 - SQLite (`meta/app.db`) for operational state
 - Parquet lake on local disk
 - Grafana service (local)
 - restic (backup)
-

Next decision (1 question)

For Grafana → backend analytics, do you prefer:

1. **Predefined endpoints only** (safer, curated queries), or
2. A **restricted "SQL-over-HTTP"** endpoint where Grafana can send limited SQL/templates?

If you answer that, I can propose the exact API shapes + a minimal dashboard set (panels, queries, and the marts you should precompute for fast rendering).

Predefined endpoints only

T



Architecture choice: Grafana calls only predefined backend endpoints

You'll run Grafana as a separate local service, but **Grafana will not query DuckDB/Parquet directly**. Instead, Grafana panels fetch data from a small set of **whitelisted analytics endpoints** exposed by your backend (which then queries DuckDB + Parquet).

This gives you:

- tight control over what can be queried,
- predictable performance,
- simpler security (no DB credentials in Grafana),
- easier future refactors of storage.

1) Component layout (single machine)

- **React app**: primary UI
- **Backend (Go/Python)**:
 - Ingest → write Parquet (raw/ , curated/ , marts/)
 - Run DuckDB transforms (scheduled)
 - Serve **predefined analytics endpoints** for Grafana
 - Reverse-proxy Grafana at /grafana

- **DuckDB**: embedded in backend (query engine)
 - **Parquet**: local-first analytical storage
 - **SQLite**: operational state (users, settings, checkpoints)
 - **Grafana**: dashboards embedded into React
-

2) Grafana integration approach (practical)

Grafana needs a way to call HTTP endpoints and plot time series. The simplest route is using a REST/JSON datasource plugin (commonly one of these):

- **Infinity** datasource, or
- **JSON API** datasource

You then configure each panel to hit a **specific endpoint** (no ad-hoc SQL). If you prefer zero plugin dependency, the alternative is building a small internal Grafana datasource plugin—but that's more work.

3) Endpoint contract (predefined + panel-friendly)

Global rules

- All endpoints require auth (same as your app session).
- Parameters are standardized:

- `from` / `to` as Unix epoch **milliseconds** (Grafana-friendly)
- `symbol` optional where relevant
- `tz` optional (default local)

Core endpoints (suggested minimal set)

A) Minute series (OHLCV)

GET `/analytics/series/minute-ohlcv?symbol=ABC&from=...&to=...`

Returns:

```
json
{
  "symbol": "ABC",
  "from": 1693526400000,
  "to": 1693612800000,
  "points": [
    { "t": 1693526460000, "o": 101.1, "h": 101.3, "l": 100.9, "c": 101.0, "v": 12345 }
  ]
}
```

B) Derived indicators (precomputed or computed-on-read)

GET `/analytics/series/indicators?symbol=ABC&from=...&to=...`

Example response:

```
json
{
  "symbol": "ABC",
  "points": [
    { "t": 1693526460000, "sma20": 101.02, "ema20": 101.01, "rv30": 0.12 }
  ]
}
```

C) Daily KPIs (fast dashboards)

GET /analytics/kpis/daily?from=...&to=...

Example:

```
json
{
  "rows": [
    { "dt": "2026-09-01", "trades": 1203, "volume": 923001, "pn1": 1532.22, "drawdown": -210.50 }
  ]
}
```

D) "Top movers" / leaders table

GET /analytics/rank/top-movers?window=60m&limit=20